PYTHON PROGRAMMING

ASSIGNMENT 1

Payment Calculator for a Village Milk Co-operative

Student Details	Information
Name	R M Varshan
Register Number	1971260030D
Course	CSA08 – Python Programming
Faculty	Dr. Pokuri Swetha
Submission Date	04 November 2026
Python Version	Python 3.12 (target environment; verify locally)

Ksheera Village Milk Producers' Co-operative

GitHub Repository: To be added after repository creation

Note: This report includes runnable code and expected/sample outputs. Actual execution screenshots, local benchmark timings and the GitHub URL must be added after running the programs on the student's computer.

CONTENTS

Introduction and project overview

Task 1 – Understand the problem and identify data

Task 2 – Explore values and types

Task 3 – Variables and constants

Task 4 – Correct expressions

Task 5 – Read input and convert types

Task 6 – Organise logic into functions

Task 7 – Build a module

Task 8 – Plan the algorithm

Task 9 – Implement and test

Task 10 – Compare expectations with execution

Task 11 – Recommend and defend the final design

Task 12 – Re-evaluate and adapt the design

Final conclusion and submission checklist

INTRODUCTION AND PROJECT OVERVIEW

The Ksheera Village Milk Producers' Co-operative collects milk from farmers twice a day. The clerk records the quantity of milk, fat percentage and SNF percentage. The co-operative calculates a rate per litre and pays farmers every ten days.

The purpose of this project is to create a reliable Python payment calculator. The program must validate entries, calculate rates consistently, avoid money errors caused by floating-point arithmetic, and support future rate revisions.

Original rate rules

Item	Rule
Base rate	₹30.00 per litre at 3.5% fat and 8.5% SNF
Fat adjustment	₹0.40 per 0.1 percentage-point difference from 3.5%
SNF adjustment	₹0.30 per 0.1 percentage-point difference from 8.5%
Payment period	Ten days
Fat validation range	2.0% to 7.0%, inclusive
Quantity	Must be greater than zero

Design decision: For the original calculator, the rate is rounded to two decimal places and each farmer's payment is rounded to two decimal places. For production use, money is represented with Decimal and values are constructed from text, not from binary float values.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE DATA (4 MARKS)

1.1 Inputs and outputs

Value	Type	Reason
Farmer ID	str	May contain letters or leading zeroes; it is an identifier, not a quantity.
Litres	Decimal	Milk quantity can contain fractional litres.
Fat percentage	Decimal	Analyser readings can contain fractional percentages.
SNF percentage	Decimal	SNF is also a fractional percentage.
Shift	str	Stores values such as Morning or Evening.
Rate per litre	Decimal	Money rate must be accurate to paisa.
Payment amount	Decimal	Money payable to one farmer.
Ten-day total	Decimal	Sum of a farmer's payments for the ten-day period.

1.2 Constants

Constant	Value	Reason
BASE_RATE	₹30.00	Standard price; centralised for easy revision.
BASE_FAT	3.5	Reference fat percentage.
BASE_SNF	8.5	Reference SNF percentage.
FAT_STEP	0.1	One adjustment step.
FAT_STEP_VALUE	₹0.40	Value of each fat step.
SNF_STEP_VALUE	₹0.30	Value of each SNF step.
MIN_FAT / MAX_FAT	2.0 / 7.0	Accepted fat limits.

1.3 Validation rules and assumptions

Litres must be numeric and greater than zero. Zero or negative quantities are rejected.

Fat must be between 2.0% and 7.0%, inclusive. The assignment does not specify an SNF range, so the program checks that SNF is numeric and non-negative; a production union should define an approved SNF range.

Farmer ID and shift cannot be blank. Shift is restricted to Morning or Evening.

A payment entry belongs to one farmer and one shift. Entries for the same farmer are accumulated over ten days.

The original rule treats each 0.1 percentage-point difference as one step. Readings are assumed to align to 0.1% in Tasks 1–11; Task 12 extends this to two decimal places.

Rate and payment are rounded to two decimal places using Decimal quantization. The rounding mode is explicitly selected as ROUND_HALF_UP for conventional half-paisa rounding.

1.4 Alternative design and complexity

For 300 farmers, using separate variables such as litres1, fat1 and snf1 would create 900 variables and repeated code. The program would need special logic for every farmer. With a list of records, each farmer's entry is stored together, for example (farmer_id, litres, fat, snf, shift). Processing n entries takes O(n) time. Storing n entries takes O(n) space. Separate named variables also require O(n) storage but do not scale cleanly because the source code grows with every farmer. A list or tuple provides a consistent structure and supports loops.

```
entries = [
```

```
    ("F001", 10, 3.5, 8.5, "Morning"),
    ("F002", 8.5, 4.2, 8.7, "Evening"),
]
for entry in entries:
    print(entry)
```

TASK 2 – EXPLORE VALUES AND TYPES IN THE INTERPRETER (4 MARKS)

2.1 Interactive session

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

The / operator performs true division. The // operator performs floor division, so -7 // 2 is -4 because floor means moving towards the next lower integer. The % operator returns the remainder. The comparison is False because

many decimal fractions cannot be represented exactly in binary floating-point.

2.2 Why Decimal is suitable for money

Float values are approximate binary numbers. Small representation errors can accumulate when thousands of payments are added. Decimal stores decimal values and supports explicit decimal rounding. Integer paise is another exact option when all amounts are represented as whole paise.

2.3 Completed TODO program

```
from decimal import Decimal



payments = ["33.40"] * 1000 + ["0.10"] * 3



def total_float(items):

    total = 0.0

    for amount in items:

        total += float(amount)

    return total



def total_decimal(items):

    total = Decimal("0.00")

    for amount in items:

        total += Decimal(amount)

    return total



print("float :", total_float(payments))

print("Decimal:", total_decimal(payments))

print(0.1 + 0.2 == 0.3,

      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected output:

```
float : 33400.30000000061

Decimal: 33400.30

False True
```

Changing Decimal(amount) to Decimal(float(amount)) first creates an approximate float and then converts that approximation to Decimal. The resulting Decimal can contain many extra digits. Decimal should be constructed directly from the original text, for example Decimal('0.10').

2.4 Alternative concepts and complexity

Method	Time per addition	Exact for money?	Comment
Decimal	O(1) for fixed precision	Yes, with text input and defined rounding	Best for decimal financial calculations.
Integer paise	O(1)	Yes, if conversion and rounding are defined	₹33.40 becomes 3340 paise.
Float + round each time	O(1)	Not guaranteed	Rounding cannot remove all binary representation issues.

sys.getsizeof(value) can be used to inspect the memory occupied by one Python object. The exact byte count depends on the Python build and platform; it should be measured on the computer used for the assignment.

```
import sys

from decimal import Decimal

print(sys.getsizeof(3340))

print(sys.getsizeof(33.40))

print(sys.getsizeof(Decimal("33.40")))
```

TASK 3 – CHOOSE VARIABLES AND NAME THE CONSTANTS (3 MARKS)

3.1 Constants and naming conventions

```
from decimal import Decimal



BASE_RATE = Decimal("30.00")

BASE_FAT = Decimal("3.5")

BASE_SNF = Decimal("8.5")

FAT_STEP = Decimal("0.1")
```

```
FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

MIN_FAT = Decimal("2.0")

MAX_FAT = Decimal("7.0")
```

Python does not enforce true constants. Uppercase names are a convention that tells programmers not to reassign a value. Variable names should be descriptive, use lowercase words separated by underscores, and begin with a letter or underscore.

3.2 Dynamic typing and assignment

```
>>> value = 10

>>> type(value)

<class 'int'>

>>> value = "ten"

>>> type(value)

<class 'str'>

>>> a, b = 5, 9

>>> a, b = b, a

>>> a, b

(9, 5)
```

Python variables refer to objects, so the same name can later refer to a different type. Multiple assignment assigns several values in one statement. Swapping uses tuple unpacking and does not require a temporary variable.

3.3 Alternative concept and complexity

If a rate such as 30.00 is typed directly in k different expressions, a revision requires changing k locations. This is a magic-number design and is error-prone. Named constants need only one change. A dictionary such as RATE_CARD = {'base': Decimal('30.00'), ...} is also suitable when a group of related values must be selected together. For a fixed number of constants, runtime and space are both O(1); the main advantage is maintainability, not speed.

TASK 4 – WRITE CORRECT EXPRESSIONS (4 MARKS)

4.1 Formulas

```
fat_steps = round((fat - BASE_FAT) / FAT_STEP)

snf_steps = round((snf - BASE_SNF) / FAT_STEP)

rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE

rate = rate.quantize(Decimal("0.01"))
```

```
amount = (litres * rate).quantize(Decimal("0.01"))
```

The step calculation uses round() because a decimal subtraction and division may produce a value such as 7.000000000000001. The code uses Decimal for money. For analyser readings, convert text to Decimal and divide by Decimal('0.1').

4.2 Unexpected expression results

```
>>> (4.2 - 3.5) / 0.1

7.000000000000001

>>> 0.1 + 0.2 == 0.3

False

>>> -7 // 2

-4

>>> int(-7 / 2)

-3

>>> round(2.5)

2
```

The first result is a binary floating-point representation effect. The second is the same representation issue. The third uses floor division, which rounds down. int() truncates towards zero, while math.floor() rounds down. Python's built-in round uses ties-to-even for exact halfway cases, which explains round(2.5) returning 2.

4.3 Fat = 4.2 and fat = 3.1 comparison

Fat	Exact step value	round()	int()	floor()
4.2	7.0	7	6	7
3.1	-4.0	-4	-3	-4

For fat 4.2, the correct adjustment is +7 steps. int() truncates 6.999999... to 6 and gives the wrong rate. For fat 3.1, the step value is -4 and all three methods give -4 in this particular case. Each operation uses O(1) time and O(1) extra space.

Operator precedence: parentheses are evaluated first, then exponentiation, unary operators and multiplication/division before addition/subtraction. Exponentiation associates from right to left. Comparison chaining such as 2.0 <= fat <= 7.0 checks both bounds. Boolean operators and, or and not combine conditions.

TASK 5 – READ INPUT AND CONVERT TYPES (3 MARKS)

input() always returns a string. Convert the text only after validating it. int() is appropriate for whole-number identifiers only when the identifier is truly numeric; IDs are safer as strings. Decimal is preferred for quantity and percentages when decimal precision matters.

```
from decimal import Decimal, InvalidOperation



def read_decimal(prompt):

    while True:

        text = input(prompt).strip()

        try:

            return Decimal(text)

        except InvalidOperation:

            print("Enter a valid number, for example 8.5.")



litres = read_decimal("Litres: ")

fat = read_decimal("Fat %: ")

snf = read_decimal("SNF %: ")

print(f"Litres: {litres:>8}  Fat: {fat:>5}%  SNF: {snf:>5}%")
```

The input '8,5' and the word 'eight' are not valid Decimal strings and raise InvalidOperation. Spaces around ' 8.5 ' are removed by strip(). A try/except conversion is generally clearer and safer than manually checking every possible numeric character pattern.

Alternative concept and complexity

String pre-checking may reject valid formats or accept invalid ones unless the validation rule is carefully written. try: Decimal(text) except InvalidOperation handles conversion directly. Both approaches take O(1) time per fixed-length entry in ordinary use; the conversion approach is simpler and handles whitespace after strip().

TASK 6 – ORGANISE THE LOGIC INTO FUNCTIONS (4 MARKS)

```
from decimal import Decimal, ROUND_HALF_UP



CENT = Decimal("0.01")



def money(value):

    """Round a monetary value to two decimal places."""
```

```
    return value.quantize(CENT, rounding=ROUND_HALF_UP)


def rate_per_litre(fat, snf):
    """Return the rate per litre for the supplied fat and SNF values."""
    fat_steps = ((fat - BASE_FAT) / FAT_STEP).to_integral_value(
        rounding=ROUND_HALF_UP)
    snf_steps = ((snf - BASE_SNF) / FAT_STEP).to_integral_value(
        rounding=ROUND_HALF_UP)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return money(rate)


def payout(litres, fat, snf):
    """Return the rounded payment for one milk entry."""
    return money(litres * rate_per_litre(fat, snf))


def ten_day_total(payments):
    """Return the sum of payment amounts for a ten-day period."""
    return money(sum(payments, Decimal("0.00")))
```

A function receives values through parameters and sends a result back using return. print() only displays a value; it does not provide the value to the calling program. Constants can be read inside functions. They should not be reassigned inside a function. Local variables exist inside their function, while global names are defined at module level.

Alternative: One long script may work for a small example but repeats calculations and makes testing harder. Lambda expressions are suitable for short expressions, not for multi-step validation and documentation. Function calls add a small fixed overhead, but processing n entries remains O(n). Reusing functions reduces repeated code and improves reliability.

TASK 7 – BUILD A MODULE (4 MARKS)

Save the following file as milkrate.py. The module contains the rate card and reusable calculation functions.

```
"""Rate card and calculations for the village milk co-operative."""

from decimal import Decimal, ROUND_HALF_UP


BASE_RATE = Decimal("30.00")
```

```
BASE_FAT = Decimal("3.5")

BASE_SNF = Decimal("8.5")

FAT_STEP = Decimal("0.1")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

CENT = Decimal("0.01")



def rate_per_litre(fat, snf):

    """Return price per litre rounded to two decimal places."""

    fat_steps = ((fat - BASE_FAT) / FAT_STEP).to_integral_value(

        rounding=ROUND_HALF_UP)

    snf_steps = ((snf - BASE_SNF) / FAT_STEP).to_integral_value(

        rounding=ROUND_HALF_UP)

    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE

    return rate.quantize(CENT, rounding=ROUND_HALF_UP)



def payout(litres, fat, snf):

    """Return payment for one entry, rounded to two decimal places."""

    return (litres * rate_per_litre(fat, snf)).quantize(

        CENT, rounding=ROUND_HALF_UP)



if __name__ == "__main__":

    for litres, fat, snf in [

        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),

        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),

        (Decimal("12"), Decimal("3.1"), Decimal("8.3"))

    ]:

        rate = rate_per_litre(fat, snf)

        pay = payout(litres, fat, snf)

        print(f"{litres:5} L fat {fat} snf {snf} "
```

```
            f"rate {rate:6.2f} pay {pay:8.2f}")
```

7.1 Separate script: main.py

```
from decimal import Decimal

import milkrate as mr



rate = mr.rate_per_litre(Decimal("4.2"), Decimal("8.7"))

payment = mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7"))

print("Rate:", rate)

print("Payment:", payment)
```

7.2 Expected output

```
10 L fat 3.5 snf 8.5 rate  30.00 pay   300.00

8.5 L fat 4.2 snf 8.7 rate  33.40 pay   283.90

12 L fat 3.1 snf 8.3 rate  27.80 pay   333.60
```

The table is not printed when milkrate is imported because the test loop is inside if __name__ == '__main__'. When the file is run directly, __name__ equals '__main__'. When imported, __name__ is 'milkrate'.

import milkrate gives access through the module name; from milkrate import payout imports one selected name; import milkrate as mr uses an alias. Python searches the current folder and directories listed in sys.path. Imported modules are cached in sys.modules. Copying functions into many scripts makes rate changes harder; importing a shared module centralises the logic. Import has a one-time loading cost, then subsequent imports use the cache.

TASK 8 – PLAN THE ALGORITHM BEFORE CODING (4 MARKS)

8.1 Problem-solving steps

Understand: identify required inputs, rate rules, outputs and validation limits.

Plan: write formulas, pseudocode and a flowchart.

Write: implement the planned logic in Python functions.

Test: check normal, boundary and invalid entries and compare results with hand calculations.

8.2 Pseudocode

```
ALGORITHM ONE_PAYMENT

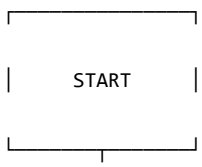
START

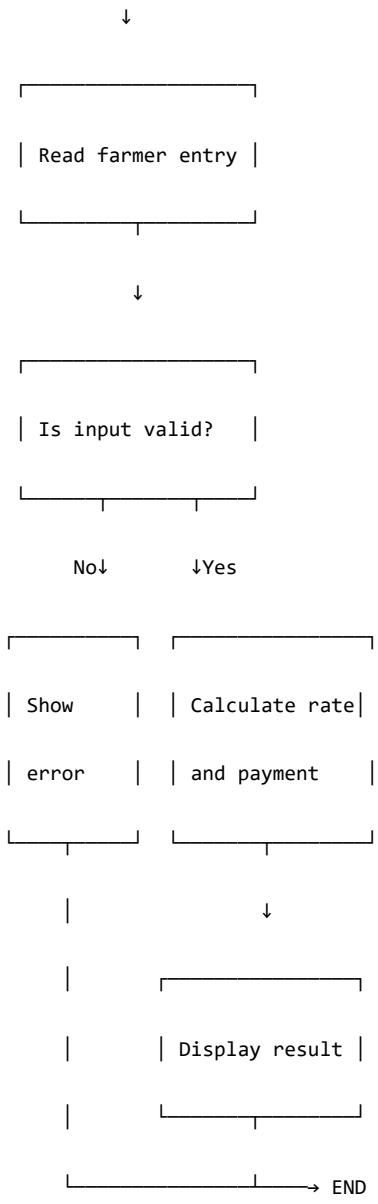
    INPUT farmer ID, litres, fat, SNF and shift
```

```
    IF farmer ID is blank THEN
        OUTPUT invalid farmer ID
    ELSE IF litres is not numeric or litres <= 0 THEN
        OUTPUT invalid quantity
    ELSE IF fat is outside 2.0 to 7.0 THEN
        OUTPUT invalid fat value
    ELSE IF SNF is not numeric or SNF < 0 THEN
        OUTPUT invalid SNF value
    ELSE
        SET fat steps TO nearest whole step from 3.5
        SET SNF steps TO nearest whole step from 8.5
        SET rate TO base rate plus fat and SNF adjustments
        SET payment TO litres multiplied by rate
        OUTPUT rate and payment
    ENDIF
END


ALGORITHM TEN_DAY_TOTAL
START
    SET total TO 0
    FOR each valid entry in the ten-day records
        SET total TO total plus payment for the entry
    ENDFOR
    OUTPUT total
END
```

8.3 Flowchart





8.4 Hand trace for three entries

Entry	Litres	Fat	SNF	Rate	Payment
F001	10.0	3.5	8.5	₹30.00	₹300.00
F002	8.5	4.2	8.7	₹33.40	₹283.90
F003	12.0	3.1	8.3	₹27.80	₹333.60
Total					₹917.50

Storing all records requires O(n) space and allows later audit, correction and reporting. A running total uses O(1) extra space and is suitable when only the final total is needed. Both methods take O(n) time. The co-operative should retain the original records for audit even if a running total is maintained.

TASK 9 – IMPLEMENT AND TEST THE CALCULATOR (4 MARKS)

9.1 Complete calculator

```
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP



BASE_RATE = Decimal("30.00")

BASE_FAT = Decimal("3.5")

BASE_SNF = Decimal("8.5")

STEP = Decimal("0.1")

FAT_VALUE = Decimal("0.40")

SNF_VALUE = Decimal("0.30")

MIN_FAT = Decimal("2.0")

MAX_FAT = Decimal("7.0")

CENT = Decimal("0.01")



def to_decimal(value):

    """Convert text or a numeric value to Decimal safely."""

    try:

        return Decimal(str(value).strip())

    except InvalidOperation:

        raise ValueError("Value must be numeric")



def rate_per_litre(fat, snf):

    """Calculate the milk rate per litre."""

    fat_steps = ((fat - BASE_FAT) / STEP).to_integral_value(

        rounding=ROUND_HALF_UP)

    snf_steps = ((snf - BASE_SNF) / STEP).to_integral_value(

        rounding=ROUND_HALF_UP)

    rate = BASE_RATE + fat_steps * FAT_VALUE + snf_steps * SNF_VALUE
```

```
    return rate.quantize(CENT, rounding=ROUND_HALF_UP)


def payout(litres, fat, snf):
    """Calculate a validated entry's payment."""
    if litres <= 0:
        raise ValueError("Litres must be greater than zero")
    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf < 0:
        raise ValueError("SNF cannot be negative")
    return (litres * rate_per_litre(fat, snf)).quantize(
        CENT, rounding=ROUND_HALF_UP)


def ten_day_total(entries):
    """Return total payment for a list of (litres, fat, snf) entries."""
    total = Decimal("0.00")
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return total.quantize(CENT, rounding=ROUND_HALF_UP)


if __name__ == "__main__":
    sample = [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3")),
        (Decimal("5"), Decimal("2.0"), Decimal("8.5")),
        (Decimal("7"), Decimal("7.0"), Decimal("8.5")),
    ]
    for row in sample:
        print(row, "=>", payout(*row))
```

```
print("Ten-day total:", ten_day_total(sample))
```

9.2 Test table

Test	Input	Expected result	Actual result
Standard	10 L, 3.5 fat, 8.5 SNF	₹300.00	Run program to confirm
Above standard	8.5 L, 4.2 fat, 8.7 SNF	₹283.90	Run program to confirm
Below standard	12 L, 3.1 fat, 8.3 SNF	₹333.60	Run program to confirm
Minimum fat	5 L, 2.0 fat, 8.5 SNF	₹120.00	Run program to confirm
Maximum fat	7 L, 7.0 fat, 8.5 SNF	₹308.00	Run program to confirm
Negative quantity	-2 L	ValueError	Run program to confirm
Fat too low	1.9%	ValueError	Run program to confirm
Invalid text	fat = 'eight'	ValueError during conversion	Run program to confirm

Expected five-entry hand total: ₹1,345.50. This value should be checked by running the program; the individual expected amounts are ₹300.00, ₹283.90, ₹333.60, ₹120.00 and ₹308.00.

9.3 Automated assertions

```
assert payout(Decimal("10"), Decimal("3.5"), Decimal("8.5")) == Decimal("300.00")
assert payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")) == Decimal("283.90")
assert payout(Decimal("12"), Decimal("3.1"), Decimal("8.3")) == Decimal("333.60")

try:
    payout(Decimal("-1"), Decimal("3.5"), Decimal("8.5"))
    assert False, "Negative litres should fail"
```

```
except ValueError:
    pass


print("All tests passed")
```

Manual testing requires repeating each input after every change. Assertions can run many checks automatically and consistently. If there are t tests, rerunning them takes O(t) time; automated tests reduce human effort and help prevent regression errors.

TASK 10 – COMPARE EXPECTATIONS WITH EXECUTION (4 MARKS)

10.1 Benchmark program

```
from decimal import Decimal
from time import perf_counter


def benchmark(n):
    # Example dataset repeated n times
    amount_text = "33.40"
    start = perf_counter()
    total_float = 0.0
    for _ in range(n):
        total_float += float(amount_text)
    float_time = perf_counter() - start


    start = perf_counter()
    total_decimal = Decimal("0.00")
    for _ in range(n):
        total_decimal += Decimal(amount_text)
    decimal_time = perf_counter() - start


    print(n, total_float, total_decimal,
          float_time, decimal_time)
```

```
for count in (300, 6000, 150000):

    benchmark(count)
```

The program should be run on the student's computer. Execution time depends on processor, operating system, Python build and background load. The report must use the measured times printed by the student's run rather than invented timings.

Number of payments	Float total (mathematical target)	Decimal total
300	₹10,020.00 (may contain a tiny binary error)	₹10,020.00
6,000	₹200,400.00 (may contain a tiny binary error)	₹200,400.00
150,000	₹5,010,000.00 (may contain a tiny binary error)	₹5,010,000.00

This repeated-payment benchmark demonstrates representation and performance. For actual bank totals, the calculator should round each farmer's payment to two decimal places according to the agreed payment policy, then sum those Decimal amounts. Rounding only the final total can produce a different result if individual entries contain fractions of a paisa.

10.2 Rate lookup alternative

A dictionary can cache the rate for each unique (fat, snf) pair. A dictionary lookup is O(1) on average. Calculating the rate directly is also O(1) per entry because it uses a fixed number of arithmetic operations. Caching can reduce repeated calculations when many entries share the same readings, but it uses O(k) extra space for k unique pairs. It is unlikely to change the overall O(n) complexity.

```
rate_cache = {}



def cached_rate(fat, snf):

    key = (fat, snf)

    if key not in rate_cache:

        rate_cache[key] = rate_per_litre(fat, snf)

    return rate_cache[key]
```

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

11.1 Recommendation

Use Decimal constructed from text for litres, fat, SNF, rate and payment. Avoid Decimal(float_value).

Use named constants or a rate-card object so that a rate revision is made in one place.

Validate farmer ID, positive quantity, fat range, non-negative SNF and a valid shift.

Round the rate and each payment to two decimal places using a documented rounding mode.

Keep transaction records for audit, while also maintaining a running ten-day total.

Separate calculations into functions and place shared logic in a module.

11.2 Scalability and limitations

A loop that processes n entries requires O(n) time. If all records are stored, memory use is O(n). The proposed design can process 150,000 entries in principle, but a real district union also needs database storage, backups, access control, audit logs, duplicate-entry prevention, concurrency handling and secure bank-export files. A single in-memory script is not a complete enterprise payment system.

11.3 Alternative: external JSON rate card

```
{
  "base_rate": "30.00",
  "base_fat": "3.5",
  "base_snf": "8.5",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30",
  "step": "0.1"
}
```

```
import json
from decimal import Decimal

with open("rate_card.json", encoding="utf-8") as file:
    raw = json.load(file)

BASE_RATE = Decimal(raw["base_rate"])
FAT_STEP_VALUE = Decimal(raw["fat_step_value"])
SNF_STEP_VALUE = Decimal(raw["snf_step_value"])
```

Reading a small JSON file adds a one-time startup cost proportional to the file size, O(m), and uses O(m) memory for m configuration values. It allows authorised staff to revise rates without editing Python source code. The file should be validated, versioned and protected from unauthorised changes.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

A. Impact analysis (2 marks)

The automatic analyser now provides fat and SNF to two decimal places. Seasonal rates apply from April through June: base rate ₹32.00 and fat step value ₹0.45. The SNF step value remains ₹0.30. Other months retain the original rate card. A farmer supplying more than 500 litres in ten days receives ₹0.50 per litre on all litres supplied. Exactly 500 litres receives no bonus.

Two-decimal readings mean the step count may be fractional if the reading is not an exact multiple of 0.1 from the reference. The co-operative must define how to handle such readings. This implementation rounds the number of 0.1% steps to the nearest whole step using ROUND_HALF_UP. This policy must be approved by the union because another policy, such as proportional adjustment, would produce different rates.

B. Evaluation of original design (2 marks)

The original functions and modular structure can be retained, but the fixed constants must be grouped into seasonal rate cards and the calculation must accept a month or season parameter. A separate bonus calculation is needed because the threshold applies to the farmer's total ten-day litres, not to an individual delivery.

C. Adaptation – revised implementation (2 marks)

```
from decimal import Decimal, ROUND_HALF_UP



CENT = Decimal("0.01")

STEP = Decimal("0.1")

BASE_FAT = Decimal("3.5")

BASE_SNF = Decimal("8.5")

SNF_STEP_VALUE = Decimal("0.30")

BONUS_RATE = Decimal("0.50")

BONUS_LIMIT = Decimal("500")



RATE_CARDS = {

    "regular": {

        "base": Decimal("30.00"),

        "fat_step": Decimal("0.40"),

    },

    "seasonal": {

        "base": Decimal("32.00"),
```

```
        "fat_step": Decimal("0.45"),
    },
}


def select_rate_card(month):
    """Select seasonal card for April-June, otherwise regular card."""
    return RATE_CARDS["seasonal"] if month in (4, 5, 6) else RATE_CARDS["regular"]


def revised_rate(fat, snf, month):
    """Calculate rate using two-decimal analyser readings and selected card."""
    card = select_rate_card(month)
    fat_steps = ((fat - BASE_FAT) / STEP).to_integral_value(
        rounding=ROUND_HALF_UP)
    snf_steps = ((snf - BASE_SNF) / STEP).to_integral_value(
        rounding=ROUND_HALF_UP)
    value = (card["base"] + fat_steps * card["fat_step"]
             + snf_steps * SNF_STEP_VALUE)
    return value.quantize(CENT, rounding=ROUND_HALF_UP)


def ten_day_payment(deliveries, month):
    """Calculate total payment and bonus for one farmer's ten-day deliveries.
    Each delivery is a tuple: (litres, fat, snf).
    """
    litres_total = sum((row[0] for row in deliveries), Decimal("0"))
    milk_total = Decimal("0.00")
    for litres, fat, snf in deliveries:
        milk_total += (litres * revised_rate(fat, snf, month)).quantize(
            CENT, rounding=ROUND_HALF_UP)

    bonus = Decimal("0.00")
```

```
    if litres_total > BONUS_LIMIT:
        bonus = (litres_total * BONUS_RATE).quantize(
            CENT, rounding=ROUND_HALF_UP)

    return (milk_total + bonus).quantize(CENT, rounding=ROUND_HALF_UP)

# Demonstration
sample = [(Decimal("300"), Decimal("4.25"), Decimal("8.63")),
          (Decimal("220"), Decimal("4.25"), Decimal("8.63"))]
print("April total:", ten_day_payment(sample, 4))
print("July total:", ten_day_payment(sample, 7))
```

D. Evidence-based defence and test cases (2 marks)

For fat 4.25%, the difference from 3.5% is 0.75%, or 7.5 steps. ROUND_HALF_UP gives 8 steps. For SNF 8.63%, the difference from 8.5% is 0.13%, or 1.3 steps; this rounds to 1 step.

Test	Season/month	Ten-day litres	Rate per litre	Bonus	Expected outcome
Seasonal rate	April (4)	300	₹32 + (8×₹0.45) + (1×₹0.30) = ₹35.90	No	₹10,770.00
Regular rate	July (7)	300	₹30 + (8×₹0.40) + (1×₹0.30) = ₹33.50	No	₹10,050.00
Bonus threshold below	April	500	₹35.90	₹0.00	No bonus
Bonus threshold above	April	520	₹0.50 × 520 = ₹260.00 bonus	₹260.00	Bonus applies to all litres

For a farmer supplying 520 litres at the same readings, the seasonal milk value is 520 × ₹35.90 = ₹18,668.00 and the bonus is ₹260.00, giving ₹18,928.00. At exactly 500 litres, no bonus is paid. These cases demonstrate that the strict greater-than condition is implemented.

The design is modified rather than fully redesigned. The modular functions, validation and Decimal approach remain useful. Seasonal rate cards make revisions centralised, and a separate ten-day bonus calculation correctly

applies the threshold to total litres. The main trade-off is that the union must approve the two-decimal step-rounding policy before deployment.

FINAL CONCLUSION

The project develops a structured Python solution for calculating milk payments at a village co-operative. It uses validated inputs, named constants, Decimal arithmetic, reusable functions, a separate module, pseudocode, testing and performance measurement. The design can be extended to seasonal rates and a ten-day volume bonus without rewriting the complete program.

For a district-level deployment, the calculation logic is suitable as a starting point, but records should be stored in a secure database and supported by audit and access-control features. All benchmark timings and test outputs should be verified by executing the submitted source files in Python 3.12.

FINAL SUBMISSION CHECKLIST

Replace the GitHub placeholder with the actual repository URL.

Create repository CSA08-1971260030D-Assignment1.

Create one folder for each task and save the relevant .py files.

Run all programs in Python 3.12 and verify outputs.

Replace 'Run program to confirm' in the test table with actual results.

Run the performance benchmark and enter the measured timings.

Capture actual screenshots for interpreter sessions and program execution.

Add README.md and requirements.txt.

Commit work progressively, with at least one commit for every task.

Upload the report and submit the GitHub link on the portal.